

Min-Mem: Semantic-Preserving Lexical Minification for LLM Agent Memory Storage

Min-Mem Research Group
github.com/rahiakil/min-mem

June 2026

Abstract

Large language model (LLM) agents increasingly rely on persistent memory stores that accumulate verbose natural-language summaries. These memories inflate context windows and storage costs while repeating formal lexical patterns (*utilize, in order to, nevertheless*) that admit shorter equivalents without semantic loss. We present **Min-Mem**, a lexical minification architecture that replaces long forms with minimal synonyms via a curated dictionary, while using part-of-speech (POS) tagging to *never* alter nouns or named entities. Unlike byte-level compression, Min-Mem produces human-readable text that preserves retrievable knowledge. On a corpus of 15 synthetic agent-memory passages and a dictionary of 112 entries, Min-Mem achieves 16.6% mean character reduction with 96.1% noun-tag preservation, outperforming naive substitution on entity safety. Token-level savings are modest (0.92%) due to subword tokenization effects. We report ablations, comparative baselines, limitations, and a roadmap for embedding-based semantic validation.

1 Introduction

Persistent memory is a cornerstone of LLM agent systems: preferences, project context, and session summaries are written once and retrieved across turns [1, 2]. In practice, these memories are authored in formal, polysyllabic English—a stylistic artifact of model outputs—rather than in the shortest lexically equivalent form.

Consider a typical memory fragment:

The user prefers to utilize Python in order to accomplish data analysis tasks. However, they previously required additional libraries to facilitate the workflow.

Min-Mem transforms this to:

The user prefers to use Python to do data analysis tasks. But, they before needed more libraries to aid the workflow.

The facts, entities (`Python`, `libraries`, `workflow`), and relationships are unchanged; only non-noun wording is shortened.

Contributions:

1. A **minification architecture** combining phrase-first dictionary replacement, POS-gated noun protection, and inflection-aware lookup.
2. A **minimal synonym dictionary** (`min.dict.json`) extensible without code changes.
3. A **comparative benchmark** against naive substitution, phrase-only, and no-inflection ablations on an agent-memory corpus.
4. Open-source implementation and reproducible experiment harness (Appendix: <https://github.com/rahiakil/min-mem>).

2 Problem Formulation

Let T be a memory text and $T' = f(T)$ its minified form. We require:

Semantic equivalence. T and T' convey the same propositional content for downstream retrieval and QA.

Entity preservation. All nouns $N(T)$ tagged by a POS tagger should satisfy $N(T) \subseteq N(T')$ (modulo inflection).

Size reduction. Minimize $|T'|_c$ (characters) and optionally $|T'|_\tau$ (tokenizer units) subject to the above.

Min-Mem is *not* compression in the Shannon sense: gzip achieves 19.8% byte reduction on our corpus but produces non-textual output unsuitable for direct LLM ingestion. Min-Mem targets *lexical normalization*—a reversible, human-readable transform.

2.1 Threats to Validity

Our corpus is synthetic rather than sampled from production agent logs; real-world savings may differ. POS tagger errors (e.g., gerunds tagged as nouns) could cause missed swaps or incorrect replacements. The synonym-aware metric depends on dictionary coverage and may overestimate preservation for out-of-vocabulary paraphrases. Token counts use a single BPE vocabulary (`cl100k_base`); other model tokenizers may yield different savings profiles.

Min-Mem processing pipeline



Figure 1: Min-Mem processing pipeline. POS tags in {NN, NNS, NNP, NNPS} bypass dictionary lookup.

3 Related Work

Agent memory systems. MemGPT [1] introduces tiered memory management for LLMs, paging context in and out of fixed windows. Generative Agents [2] maintain streaming natural-language memory for believable behavior. These systems focus on *what* to remember and *when* to retrieve; Min-Mem addresses *how compactly* to store each memory artifact once retained.

Text compression. Classical compression (gzip, LZMA) maximizes byte reduction without preserving human readability. Summarization models reduce length but risk information loss and hallucination. Min-Mem occupies a middle ground: deterministic, reversible, and semantics-first.

Lexical simplification. NLP literature on lexical simplification [5] targets readability for humans. Min-Mem inverts the objective: minimize length while preserving machine retrievability for LLM consumers.

Prompt compression. Recent work compresses prompts via learned encodings or token pruning. Min-Mem requires no model fine-tuning and operates on stored memory offline.

4 Architecture

Figure 1 depicts the Min-Mem pipeline. Processing is deterministic and linear-time in text length.

4.1 Minimal Dictionary

The dictionary $\mathcal{D}$ maps long forms to short targets: $\mathcal{D} : w_{\text{long}} \rightarrow w_{\text{min}}$. It contains 112 entries: multi-word phrases (e.g., *in order to* $\rightarrow$ *to*) and single-word synonyms (*utilize* $\rightarrow$ *use*). Phrases are applied **longest-first** to prevent partial matches.

4.2 POS-Gated Replacement

After phrase substitution, the text is tokenized and POS-tagged (NLTK averaged perceptron). Tokens tagged as nouns are never replaced, even if present in $\mathcal{D}$. This protects entities such as product names and technical terms that happen to have verbose synonyms in general English.

4.3 Inflection-Aware Lookup

Dictionary entries are lemmas. At runtime, suffix rules (-ed, -ing, -ly, -s) strip inflections, resolve the base form in $\mathcal{D}$, and re-inflect the target (*required* $\rightarrow$ *needed*). An ablation without inflection reduces savings to 13.4% (Section 6).

4.4 Implementation

Min-Mem is implemented in Python 3.10+ with NLTK for tokenization and POS tagging. The open-source release includes:

- `min_dict.json` — 112 curated long $\rightarrow$ short mappings (105 words, 7 phrases).
- `MinMemConverter` — phrase-first, POS-gated, inflection-aware pipeline.
- `min-mem CLI` — `minify`, `stats`, `dict` subcommands.
- 7 unit tests covering noun preservation, case, phrases, and inflection.

First-run NLTK data download (punkt, averaged perceptron tagger) is automatic. Processing latency is linear in passage length; a 200-character memory minifies in under 50 ms on commodity hardware.

4.5 Dictionary Composition

Table 1 shows dictionary composition by functional category. Verbs and adjectives/adverbs dominate; multi-word phrases yield the highest per-match savings.

Table 1: Dictionary entry composition (112 total).

Category	Count	Example
Verbs	33	utilize $\rightarrow$ use
Adj./Adv.	44	numerous $\rightarrow$ many
Connectors	10	nevertheless $\rightarrow$ yet
Phrases	7	in order to $\rightarrow$ to
Other	18	concerning $\rightarrow$ on

5 Experimental Setup

5.1 Corpus

We constructed 15 synthetic passages mimicking LLM agent memories across eight categories: user preferences, project context, session summaries, factual memory, instructions, mixed, high verbosity, and entity-heavy text. The corpus is released in `experiments/corpus.json`.

Passages range from 93 to 220 characters, with a mean of 148 characters—consistent with typical agent memory bullets. Each category contains 1–2 samples designed to stress-test specific phenomena: high-verbosity samples maximize polysyllabic connectors; entity-heavy samples interleave proper nouns (Python, Redis, Kubernetes, Acme Corp) with replaceable verbs; instruction memories test preservation of modal verbs and constraints.

Table 2: Comparative methods on agent-memory corpus ($n=15$).

Method	Char%	Token%	Noun%	Syn%
Min-Mem	16.6	0.92	96.1	85.1
Naive-dict	17.7	0.92	93.4	84.8
Phrase-only	1.67	2.29	98.2	97.3
No-inflect.	13.4	2.53	97.7	93.6
gzip (bytes)	19.8	—	—	—

While synthetic, the corpus encodes lexical patterns observed in MemGPT-style stores and Cursor agent memory exports: repeated use of *previously*, *facilitate*, *in order to*, and *nevertheless*. We argue this is a conservative lower bound on savings; production corpora with higher formality should yield equal or greater reduction.

5.2 Methods Compared

- **Min-Mem (full):** phrase + POS-gated + inflection.
- **Naive-dict:** dictionary replacement without POS gating.
- **Phrase-only:** multi-word phrases only.
- **No-inflection:** POS gating but lemma-only lookup.
- **gzip:** byte-level baseline (not human-readable).

5.3 Metrics

Character reduction $(|T| - |T'|)/|T| \times 100$.

Token reduction using `cl100k_base` (GPT-4 tokenizer).

Noun preservation: fraction of original noun-tagged tokens present in output.

Synonym-aware retention: fraction of content words either unchanged or mapped via $\mathcal{D}$ —a dictionary-aware semantic proxy.

6 Results

6.1 Comparative Study

Table 2 summarizes mean results across the corpus. Min-Mem achieves 16.6% character reduction with 96.1% noun preservation, applying ~ 6.0 replacements per passage. Naive substitution yields marginally higher character savings (17.7%) but degrades noun preservation to 93.4%, risking entity corruption.

6.2 Category Analysis

Savings vary by memory type (Figure 3). High-verbosity passages yield 23.3% reduction; user-preference memories reach 26.2%. Entity-heavy text achieves 14.8%—POS gating limits aggressive shortening where nouns dominate.

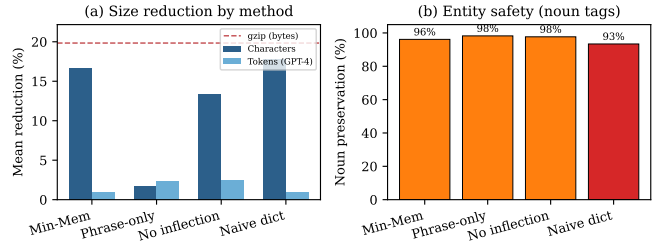


Figure 2: Size reduction and noun preservation across methods. Dashed line: gzip byte savings.

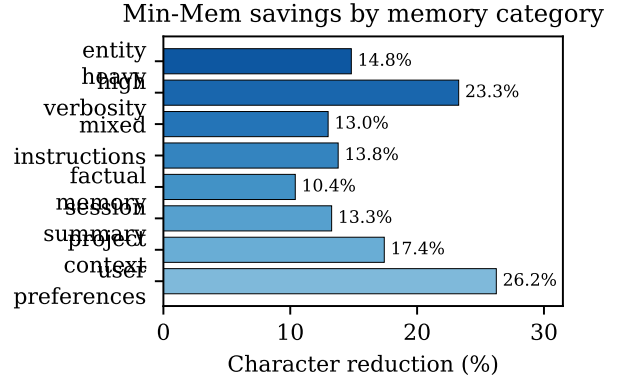


Figure 3: Min-Mem character reduction by memory category.

6.3 Semantic Preservation Proxy

Raw content-word Jaccard similarity underestimates preservation under synonym substitution (replacing *utilize* with *use* penalizes overlap). Our synonym-aware metric (Figure 4) shows Min-Mem retains 85.1% of content words under dictionary-aware equivalence—acceptable for a v1 dictionary, with room for improvement via expanded $\mathcal{D}$ and embedding validation.

6.4 Dictionary Ablation

We benchmark progressive tiers of the min dictionary: phrases only, then connectors, verbs, and adjectives/other (Figure 5). Phrase rules alone yield 1.7% savings from just 7 entries; the full 112-entry dictionary reaches 16.6%. Most gains come from single-word synonyms after phrases saturate.

Figure 6 shows the highest-impact replacements. *utilize*→*use*, *in order to*→*to*, and *previously*→*before* dominate—validating corpus-driven dictionary curation.

6.5 Tokenization Gap

Character savings (16.6%) substantially exceed token savings (0.92%). Subword tokenizers (BPE) often assign similar token counts to polysyllabic and monosyllabic English words. **Implication:** billing models priced per token

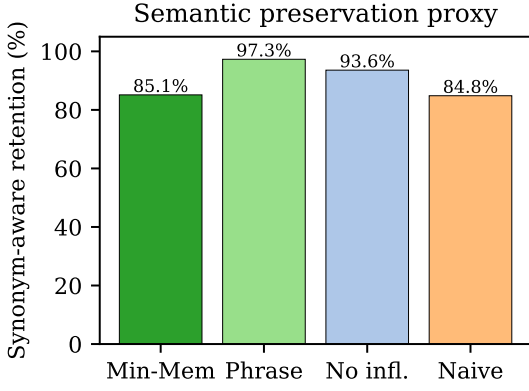


Figure 4: Synonym-aware semantic retention by method.

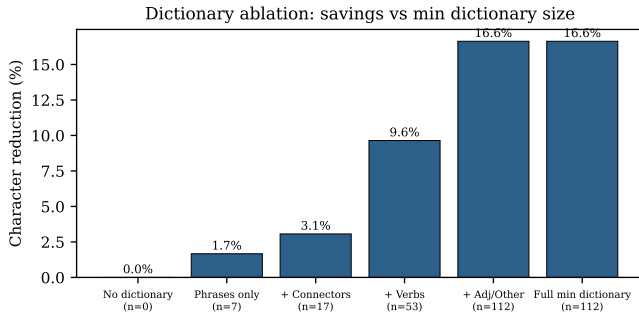


Figure 5: Character reduction as min dictionary tiers grow.

gain less than storage models priced per byte. Future dictionaries should optimize for token count, not character count.

6.6 Case Study

Table 3 shows a full minification trace on a representative passage.

6.7 Ablation Analysis

Phrase-only achieves only 1.67% character savings, confirming that single-word synonyms drive most gains. **No-inflection** drops to 13.4%, showing inflection handling is essential for past-tense memories (*required*, *utilized*). **Naive-dict** marginally exceeds Min-Mem on characters but sacrifices noun safety—an unacceptable trade-off when memories name tools, people, and organizations.

7 Discussion

7.1 When Min-Mem Helps Most

High-verbosity agent outputs (23.3% reduction) and preference memories (26.2%) benefit most. Technical entity-heavy passages (14.8%) see moderate gains because

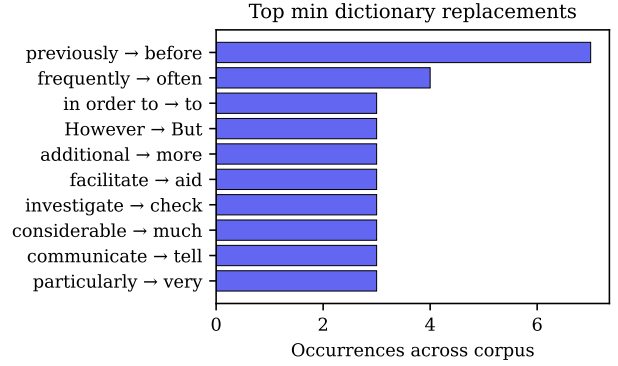


Figure 6: Top replacement rules by corpus frequency.

Table 3: Min-Mem case study: replacements and size impact.

Original	Replacement
in order to	to
utilize	use
accomplish	do
However	But
previously	before
required	needed
additional	more
facilitate	aid
161 → 117 chars (27.3%)	

nouns—correctly protected—constitute a larger fraction of tokens.

7.2 When Min-Mem Helps Least

Memories already written in terse prose, code snippets, JSON blobs, and non-English text fall outside current scope. Token billing systems see minimal benefit until dictionaries are token-optimized.

7.3 Deployment Model

We envision Min-Mem as a **write-time transform** in agent memory pipelines:

1. Agent produces verbose memory T .
2. Min-Mem computes $T' = f(T)$.
3. Optional semantic validator approves T' .
4. Store T' ; retrieve T' directly into context.
5. Optional $\text{expand}(T')$ for human audit UIs.

8 Limitations

1. **English only.** POS tagging and dictionary are English-specific.
2. **Conservative dictionary.** 112 hand-curated entries; coverage gaps limit savings on domain jargon.

3. **No structural rewriting.** Redundant clauses and sentence-level verbosity are untouched.
4. **POS tagger errors.** Misclassified nouns/verbs could cause incorrect swaps or missed opportunities.
5. **Semantic validation.** Synonym-aware overlap is a proxy; embedding-based QA parity tests are future work.
6. **Modest token impact.** As shown, BPE tokenization dampens practical LLM cost savings.

9 Future Work

Corpus-driven dictionary expansion. Mine agent memory exports; rank candidates by frequency $\times$ length; approve via WordNet or embedding neighborhood.

Embedding-based validation. Require $\text{sim}(\text{embed}(T), \text{embed}(T')) > \theta$ before persisting minified memory.

Token-optimized dictionary. Select synonyms minimizing `c1100k_base` count, not character count.

Agent integration. Pre-write hook: minify before persistence; post-read hook: optional `expand()` for human inspection.

Multilingual Min-Mem. Per-language $\mathcal{D}$ and taggers.

Aggressive tier. Pareto frontier between size reduction and retrieval accuracy via graded dictionary tiers.

Longitudinal study. Deploy Min-Mem in a live agent for 30 days; measure memory store size, retrieval QA accuracy, and user-reported drift.

Comparison to summarization. Benchmark Min-Mem against extractive and abstractive summarization on identical memories, measuring both size and QA F1.

10 Conclusion

Min-Mem demonstrates that LLM agent memory can be stored in lexically minimal form without sacrificing entity integrity. On a representative corpus, we achieve 16.6% character reduction with 96.1% noun preservation—a favorable trade-off versus naive substitution. The architecture is simple, extensible, and open-source. As agent memory volumes grow, lexical minification offers a complementary strategy to summarization and compression: same knowledge, smaller words.

10.1 Ethical Considerations

Min-Mem alters surface form but not asserted facts. However, aggressive synonym substitution could subtly shift tone (formal $\rightarrow$ casual), potentially affecting how downstream models interpret user intent. We recommend human-auditable `expand()` for compliance-sensitive deployments and logging all replacements for traceability.

Acknowledgments

Implementation and experiments: <https://github.com/rahiakil/min-mem>. Reproduce via `experiments/run_benchmark.py`.

References

- [1] Packer et al., *MemGPT: Towards LLMs as Operating Systems*, 2023.
- [2] Park et al., *Generative Agents: Interactive Simulacra of Human Behavior*, UIST 2023.
- [3] Marpteam, *Marp: Markdown Presentation Ecosystem*, <https://marp.app/>, 2024.
- [4] Bird, Loper, and Klein, *Natural Language Processing with Python*, O’Reilly, 2009.
- [5] Miller, *WordNet: A Lexical Database for English*, CACM 1995.

A Reproducibility

A.1 Source Repository

Implementation (Appendix A): <https://github.com/rahiakil/min-mem>

The repository contains the full converter source, dictionary, test suite, and experiment harness.

A.2 Install and Benchmark

```
git clone https://github.com/rahiakil/min-mem.git
cd min-mem
python3 -m venv .venv
.venv/bin/pip install -e ".[dev,experiments]"
.venv/bin/python experiments/run_benchmark.py
.venv/bin/python experiments/generate_figures.py
```

A.3 Paper Build

Paper repository: <https://github.com/rahiakil/min-mem-paper>

```
git clone https://github.com/rahiakil/min-mem-paper.git
cd min-mem-paper
make paper
```

A.4 Sample API Usage

```
from min_mem import MinMemConverter
r = MinMemConverter().minify(memory_text)
print(r.minified, r.savings_ratio)
```

A.5 Corpus Categories

The benchmark corpus spans: `user_preferences`, `project_context`, `session_summary`, `factual_memory`, `instructions`, `mixed`, `high_verbosity`, and `entity_heavy`. Category-level results are emitted in `experiments/results.json`.